

You are a senior QA engineer and TDD architect. We are building CareOrbit using:

Layer	Choice
Backend	Python 3.12 + FastAPI 0.111
Frontend	React 18 + Vite 5
Monorepo	Turborepo (web + future mobile)
Styling	Tailwind CSS 3.4 + NativeWind ready
State	Zustand + TanStack Query v5
Database	Azure PostgreSQL Flexible Server + pgcrypto
All 10 Azure services	As decided earlier

DECISION 3 — Azure Services

Here is every service, what it does specifically in CareOrbit, why it was chosen over alternatives, the exact package, and the mock path for tests.

#	Service	What It Does in CareOrbit	Why This Over Alternatives	Package	Test Mock Path
---	---------	---------------------------	----------------------------	---------	----------------

1	Azure AI Document Intelligence	OCR on prescription photos, medicine strips, lab reports, extracts structured tables, handwriting, printed text	Chosen over Computer Vision Read API because the prebuilt -document model understands medical document layout (lab report tables, key-value pairs like "HbA1c: 7.8%") natively	azure-ai-document-intelligence==1.0.0	services.azure_vision.AzureVisionService.extract_text
2	Azure Open AI (GPT-4o)	Structured medical information extraction from OCR text, care gap reasoning,	GPT-4o over GPT-3.5: Hindi abbreviations (BD/OD/H S/SOS), brand-to-generic mapping (Thyronor	openai==1.35.0	services.azure_openai.AzureOpenAIService.extract_structured_data

multi-agent
responses
e
synthes
is, Hindi
medical
term
underst
anding

m→Levoth
yroxine),
multi-hop
reasoning
for PHIG

3 **Azure AI
Translator**

Hindi ↔
English
for
patient
queries
and AI
responses

No
alternative
— only
service
with Indian
language
support at
this quality
level. Uses
httpx
directly (no
async
Azure
Translator
SDK
exists)

httpx==0
.27.0

services.azure_trans
lator.httpx.AsyncCli
ent

4 **Azure AI
Language
(Healthcar**

Extract
medication
names,
dosage
s,
diagnosis

Chosen
over
generic
NER
because it
returns
dataSou

azure-ai
-textana
lytics==
5.3.0

services.azure_langu
age.TextAnalyticsCli
ent

es from `ceEntity`
NER) OCR `Id` pointing
text directly to
with `RxNorm`
direct codes —
`RxNorm` eliminates
m and the
ICD-10 fuzzy-match
entity h step for
linking well-structu
red text

5 `Azure` Store Required `azure-st` `services.azure_blob.`
`Blob` upload — no `orage-bl` `BlobServiceClient`
`Storage` d alternative. `ob==12.2`
`ge` prescrip PDFs can't `0.0`
tion live in
images; Postgres
serve (size +
generat streaming)
ed
health
summa
ry
PDFs
via
time-lim
ited
SAS
URLs

6 `Azure` Medicat Chosen `azure-co` `services.azure_email`
`Com` ion over `municat` `.EmailClient.begin_s`
`muni` remind `SendGrid:` `end`
er single

cation n Services (Email)	emails, upload result notification ions, drug interaction on alert emails	Azure billing, HIPAA-eligible service tier, begin_send() returns a poller (async-friendly)	ion-email==1.0.0
------------------------------------	---	---	------------------

7 Azure AI Search	RAG for drug interaction lookup (10K+ drug pairs indexed with vectors) and clinical guideline retrieval for care gap detection	Chosen over Pinecone/Weaviate: hybrid keyword+vector search catches both spelling variants of drug names AND semantic similarity. Stays in Azure billing	azure-search-documents==11.6.0	services.azure_search.SearchClient.search
-------------------	--	--	--------------------------------	---

8	Azure PostgreSQL Flexible Server All relational data — users, PHIG nodes/edges, audit_log, reminders, subscriptions, usage tracking	Chosen over Cosmos DB: pgcrypto extension for pgp_sym_encrypt PII encryption at rest, ACID transactions for append-only audit_log, HIPAA BAA available	<pre> asyncpg=0.29.0 + sqlalchemy==2.0.30 </pre>	Real DB in Docker for integration tests (pytest-asyncio)
---	---	--	--	--

9	Azure Key Vault ENCRYPTION_KEY (for pgcrypto), JWT_SECRET, all Azure API keys	Required for production: secret rotation without redeployment, audit trail for every key access, HIPAA	<pre> azure-keyvault-secrets==4.8.0 </pre>	<pre> services.azure_keyvault.SecretClient.get_secret </pre>
---	---	--	--	--

		requiremen	
		t	
1	Azur	Hosting	First-class
0	e	Next.js	Next.js
	Static	fronten	support,
	Web	d	built-in
	Apps		staging
			slots,
			global
			CDN, free
			tier
			appropriate
			for
			demo/com
			petition

Here is every service, what it does specifically in CareOrbit, why it was chosen over alternatives, the exact package, and the mock path for tests.

PROMPT 3 OF 3 — FULL TDD BUILD WITH AZURE SERVICE INTEGRATION

(Paste ONLY after human has reviewed and approved all test cases)

=====

You are a senior full-stack engineer. Human has reviewed and approved all test cases.

Now implement CareOrbit following strict TDD: for each Sprint, write failing tests first,

then write the minimum code to make them pass, then refactor.

APPROVED TEST SUITE: All 23 test files from Prompt 2 are in /tests/
STACK CONFIRMED:

Layer	Choice
Backend	Python 3.12 + FastAPI 0.111
Frontend	React 18 + Vite 5
Monorepo	Turborepo (web + future mobile)

DB: Azure PostgreSQL Flexible Server + pgcrypto extension

Cache: Redis (Azure Cache for Redis) — rate limiting + session

Queue: Azure Service Bus — async reminder email delivery

AZURE SERVICE INTEGRATION INSTRUCTIONS (apply to every service)

1. AZURE AI VISION (Document Intelligence)

Package: azure-ai-documentintelligence==1.0.0

Why this over Computer Vision: prebuilt-document model understands medical document layout (tables, key-value pairs, handwriting).

Async polling is critical — OCR takes 3-8 seconds.

Implementation pattern:

```
```python
services/azure_vision.py
```

```

from azure.ai.documentintelligence.aio import
DocumentIntelligenceClient
from azure.core.credentials import AzureKeyCredential

class AzureVisionService:
 def __init__(self):
 self.client = DocumentIntelligenceClient(
 endpoint=settings.AZURE_VISION_ENDPOINT,
 credential=AzureKeyCredential(settings.AZURE_VISION_KEY)
)

 async def extract_text(self, image_bytes: bytes) -> OCRResult:
 async with self.client:
 poller = await self.client.begin_analyze_document(
 "prebuilt-read",
 analyze_request=image_bytes,
 content_type="image/jpeg"
)
 result = await poller.result()
 full_text = "\n".join(
 line.content
 for page in result.pages
 for line in (page.lines or [])
)
 avg_confidence = sum(
 word.confidence
 for page in result.pages
 for line in (page.lines or [])
 for word in (line.words or [])
 if word.confidence
) / max(1, sum(
 len(line.words or [])
 for page in result.pages
 for line in (page.lines or [])
))

```

```

 return OCRResult(
 full_text=full_text,
 lines=[l.content for p in result.pages for l in (p.lines or [])],
 avg_confidence=avg_confidence,
 page_count=len(result.pages)
)
'''

```

```

Test mock pattern:
'''python
In conftest.py
@pytest.fixture
def mock_vision():
 service = AsyncMock(spec=AzureVisionService)
 mock_result = MagicMock()
 mock_result.pages = [MagicMock()]
 mock_result.pages[0].lines = [
 MagicMock(content="Tab Metformin 500mg BD", words=[
 MagicMock(content="Tab", confidence=0.97),
 MagicMock(content="Metformin", confidence=0.95),
])
]

 service.extract_text = AsyncMock(return_value=OCRResult(
 full_text="Tab Metformin 500mg BD",
 lines=["Tab Metformin 500mg BD"],
 avg_confidence=0.94,
 page_count=1
))
 service.classify_document_type =
 AsyncMock(return_value="prescription")
 return service
'''

```

CI/CD note: In GitHub Actions, use Docker with azurite (Azure emulator) for Blob Storage. Vision and OpenAI require real Azure credentials in

integration tests — use GitHub Secrets, never committed .env.

## 2. AZURE OPENAI SERVICE (GPT-4o)

Package: openai==1.35.0 (Azure OpenAI uses same package since v1.0)

Why GPT-4o over GPT-3.5: medical entity extraction requires context understanding for Hindi/English drug names, dosage abbreviations (BD=twice daily).

Implementation pattern:

```
```python
# services/azure_openai.py
from openai import AsyncAzureOpenAI

class AzureOpenAIService:
    def __init__(self):
        self.client = AsyncAzureOpenAI(
            azure_endpoint=settings.AZURE_OPENAI_ENDPOINT,
            api_key=settings.AZURE_OPENAI_API_KEY,
            api_version="2024-02-01"
        )
        self.deployment = settings.AZURE_OPENAI_DEPLOYMENT #
        "gpt-4o"

    async def extract_structured_data(
        self, ocr_text: str, doc_type: str
    ) -> dict:
        response = await self.client.chat.completions.create(
            model=self.deployment,
            messages=[
                {"role": "system", "content":
EXTRACTION_PROMPT[doc_type]},
                {"role": "user", "content": ocr_text}
            ],
            response_format={"type": "json_object"},
            temperature=0.1, # Low temperature for medical extraction
```

```

        max_tokens=2000
    )
    return json.loads(response.choices[0].message.content)
'''

```

```

Test mock pattern:
'''python
@pytest.fixture
def mock_openai():
    service = AsyncMock(spec=AzureOpenAIService)
    service.extract_structured_data = AsyncMock(return_value={
        "medications": [
            {"name": "Metformin", "dosage": "500mg", "frequency": "BD",
             "drug_match_score": 0.98, "needs_confirmation": False}
        ],
        "doctor_name": "Dr. Amit Roy",
        "diagnoses": ["Type 2 Diabetes Mellitus"],
        "date": "15/01/2026"
    })
    service.chat = AsyncMock(return_value="Mocked AI response")
    return service
'''

```

Critical for testing: Always mock at the service class level, not the `openai.AsyncAzureOpenAI` level. This ensures tests verify your service's response contract, not OpenAI's SDK internals.

3. AZURE AI TRANSLATOR

Package: Use `requests/httpx` directly (no official async Azure Translator SDK)

Why `httpx` over `requests`: native async support, same API surface.

```

Implementation pattern:
'''python
# services/azure_translator.py

```

```

import httpx

class AzureTranslatorService:
    BASE_URL = "https://api.cognitive.microsofttranslator.com"

    async def translate(self, text: str, target: str, source: str = None) -> str:
        params = {"api-version": "3.0", "to": target}
        if source:
            params["from"] = source
        async with httpx.AsyncClient() as client:
            response = await client.post(
                f"{self.BASE_URL}/translate",
                params=params,
                headers={
                    "Ocp-Apim-Subscription-Key":
settings.AZURE_TRANSLATOR_KEY,
                    "Ocp-Apim-Subscription-Region":
settings.AZURE_TRANSLATOR_REGION,
                },
                json=[{"Text": text}],
                timeout=10.0
            )
            return response.json()[0]["translations"][0]["text"]
'''

```

Test mock: patch("services.azure_translator.httpx.AsyncClient") — mock the entire httpx client to avoid real network calls.

4. AZURE AI LANGUAGE (Healthcare NER)

Package: azure-ai-textanalytics==5.3.0

Why: Recognize medical entities with RxNorm/ICD-10 links directly from text.

Implementation pattern:

```

```python
from azure.ai.textanalytics.aio import TextAnalyticsClient

 async def recognize_health_entities(self, text: str) -> list[MedicalEntity]:
 async with TextAnalyticsClient(
 self.endpoint, AzureKeyCredential(self.key)
) as client:
 poller = await client.begin_analyze_healthcare_entities([text])
 result = await poller.result()
 entities = []
 async for doc in result:
 for entity in doc.entities:
 entities.append(MedicalEntity(
 text=entity.text,
 category=entity.category,
 confidence=entity.confidence_score,
 rxnorm_id=next(
 (l.id for l in entity.data_sources
 if l.name == "RxNorm"), None
)
))
 return entities
```

```

5. AZURE BLOB STORAGE

Package: azure-storage-blob==12.20.0

Use async client for all operations.

Implementation pattern:

```

```python
from azure.storage.blob.aio import BlobServiceClient

 async def upload_document(self, image_bytes: bytes, patient_id: str,
doc_id: str) -> str:
 async with BlobServiceClient.from_connection_string(

```

```

 settings.AZURE_STORAGE_CONNECTION_STRING
) as client:
 container = client.get_container_client("prescriptions")
 blob_name = f"{patient_id}/{doc_id}.jpg"
 await container.upload_blob(blob_name, image_bytes,
overwrite=True)
 # Generate time-limited SAS URL (1 hour)
 sas_token = generate_blob_sas(
 account_name=client.account_name,
 container_name="prescriptions",
 blob_name=blob_name,
 account_key=settings.AZURE_STORAGE_KEY,
 permission=BlobSasPermissions(read=True),
 expiry=datetime.utcnow() + timedelta(hours=1)
)
 return
f"https://{client.account_name}.blob.core.windows.net/prescriptions/{blob_n
ame}?{sas_token}"
'''

```

## 6. AZURE COMMUNICATION SERVICES (Email)

Package: azure-communication-email==1.0.0

Implementation pattern:

```

```python
from azure.communication.email import EmailClient

async def send_medication_reminder(
    self, to_email: str, patient_name: str,
    medication_name: str, dosage: str, time_label: str
) -> bool:
    client = EmailClient.from_connection_string(
        settings.AZURE_COMM_CONNECTION_STRING
    )
    message = {

```

```

    "senderAddress": settings.AZURE_COMM_SENDER_EMAIL,
    "recipients": {"to": [{"address": to_email}]},
    "content": {
        "subject": f"CareOrbit Reminder: {medication_name} —
{time_label}",
        "plainText": REMINDER_TEMPLATE_TEXT.format(
            name=patient_name, med=medication_name, dose=dosage
        ),
        "html": REMINDER_TEMPLATE_HTML.format(
            name=patient_name, med=medication_name, dose=dosage
        )
    }
}

poller = client.begin_send(message)
result = poller.result()
return result.status == "Succeeded"
'''

```

7. AZURE AI SEARCH (RAG for Drug Interactions + Guidelines)

Package: azure-search-documents==11.6.0

Index design:

- drug_interactions index: drug_a, drug_b, severity, description, clinical_action, severity_modifiers (JSON), embedding (text-embedding-3-small)
- clinical_guidelines index: condition, screening_type, frequency, evidence_grade, source, applicable_codes (ICD-10 array), embedding

Implementation pattern:

```

'''python
from azure.search.documents.aio import SearchClient
from azure.search.documents.models import VectorizedQuery

async def search_drug_interactions(
    self, drug_a: str, drug_b: str

```

```

) -> list[DrugInteraction]:
    async with SearchClient(
        self.endpoint, "drug-interactions",
        AzureKeyCredential(self.search_key)
    ) as client:
        # Hybrid search: keyword + vector
        embedding = await self.get_embedding(f"{drug_a} {drug_b}
interaction")
        vector_query = VectorizedQuery(
            vector=embedding, k_nearest_neighbors=3,
            fields="embedding"
        )
        results = await client.search(
            search_text=f"{drug_a} {drug_b}",
            vector_queries=[vector_query],
            filter=f"drug_a eq '{drug_a}' or drug_b eq '{drug_a}'",
            top=5
        )
        return [
            DrugInteraction(**result)
            async for result in results
            if result["@search.score"] > 0.7
        ]
'''

```

```

=====
=====
SPRINT 1 BUILD — INFRASTRUCTURE + AUTH (TDD)
=====
=====
=====

```

Step 1 (FAIL): Run tests/unit/test_auth_tokens.py → all RED

Step 2 (CODE): Implement api/middleware/auth.py (JWT, bcrypt, create_refresh_token)

Step 3 (PASS): Run test_auth_tokens.py → all GREEN

Step 4 (FAIL): Run tests/functional/test_api_auth.py → all RED

Step 5 (CODE): Implement api/routes/auth.py (register, login, refresh, logout)

- register: check duplicate email, hash password, INSERT with pgp_sym_encrypt,

- create refresh token, return access_token + refresh_token immediately

- (NO OTP in Phase 1 — Phase 2 gate exists in test_api_otp.py @phase2)

- login: verify password with bcrypt, check is_active, return tokens

- refresh: verify token SHA-256 hash, check not revoked, rotate token

- logout: revoke specific token by hash

Step 6 (PASS): Run test_api_auth.py → all GREEN

POSTGRES SCHEMA — create in database/schema.sql:

```
```sql
```

```
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
```

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto"; -- REQUIRED for pgp_sym_encrypt
```

```
CREATE TABLE users (
```

```
 id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
 name TEXT NOT NULL,
```

```
 -- PII ENCRYPTED at rest:
```

```
 email TEXT NOT NULL, -- stored as pgp_sym_encrypt(email, :encryption_key)
```

```
 phone_number TEXT, -- stored as pgp_sym_encrypt(phone, :encryption_key)
```

```
 password_hash TEXT NOT NULL,
```

```
 date_of_birth DATE,
```

```
 gender VARCHAR(10),
```

```
 preferred_language VARCHAR(5) DEFAULT 'en',
```

```
 subscription_tier subscription_tier DEFAULT 'free',
```

```
 is_active BOOLEAN DEFAULT TRUE,
```

```
 created_at TIMESTAMPTZ DEFAULT NOW(),
```

```
 updated_at TIMESTAMPTZ DEFAULT NOW())
```

```
);
```

```
CREATE TABLE refresh_tokens (
 id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
 user_id UUID NOT NULL REFERENCES users(id) ON DELETE
 CASCADE,
 token_hash VARCHAR(64) NOT NULL UNIQUE, -- SHA-256 of raw
 token
 ip_address INET,
 user_agent TEXT,
 expires_at TIMESTAMPTZ NOT NULL,
 revoked_at TIMESTAMPTZ,
 created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- audit_log: APPEND ONLY — no UPDATE/DELETE permissions for app
user
```

```
CREATE TABLE audit_log (
 id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
 user_id UUID NOT NULL,
 patient_id UUID NOT NULL,
 action VARCHAR(50) NOT NULL,
 resource_type VARCHAR(50),
 resource_id UUID,
 ip_address INET,
 user_agent TEXT,
 metadata JSONB DEFAULT '{}',
 created_at TIMESTAMPTZ DEFAULT NOW()
 -- NO updated_at — this table is append-only by design
);
```

```
```
```

```
SPRINT 2 BUILD — DOCUMENT PIPELINE (TDD)
```

Step 1 (FAIL): Run tests/integration/test_document_state_machine.py → all RED

Step 2 (CODE): Implement pipeline/document_pipeline.py with these rules:

- process_document() ALWAYS returns terminal state (success | needs_confirmation | failed)
- "processing" is an internal state NEVER returned to callers
- Vision timeout → failed with error_message
- Any OCR confidence < 0.30 → document_type="unreadable", status="failed"
- Med confidence < threshold AND not critical → needs_confirmation item

Step 3 (PASS): Run test_document_state_machine.py → all GREEN

Step 4 (FAIL): Run tests/functional/test_api_documents.py → all RED

Step 5 (CODE): Implement api/routes/documents.py with:

- Allowed MIME types: image/jpeg, image/png, image/webp, image/heic
- Size limit: 10MB (check file.size before reading bytes)
- Feature gate: await enforce_document_limit(current_user["id"])
- Processing priority: 2-second yield for free tier

Step 6 (PASS): Run all document tests → GREEN

SPRINT 3 BUILD — INTERACTION ENGINE (TDD)

Step 1 (FAIL): Run tests/false_positive_negative/ → all RED

NOTE: These are split into WIRING (mock pipeline) and ALGORITHM (mock only Azure Search)

The algorithm tests run the real phig_builder detection logic.

They will fail because phig_builder doesn't exist yet.

Step 2 (CODE): Implement graph/phig_builder.py:

- `check_interactions_for_node()`: calls Azure Search, applies `severity_modifiers`
- Severity modifiers: `renal_impairment` escalates Moderate → HIGH if `creatinine > 1.3`
- `evaluate_care_gaps()`: RAG lookup against `clinical_guidelines` index

Step 3 (PASS): Wiring tests pass (mocked pipeline). Algorithm tests pass (real engine, mocked Search).

SPRINT 4 BUILD — AI CHAT + ORCHESTRATOR (TDD)

Step 1 (FAIL): Run `tests/integration/test_orchestrator.py` → all RED

Step 2 (CODE): Implement `agents/orchestrator.py`:

- Keyword routing: `MEDICATION_KEYWORDS` → `MedicationAgent`, `CARE_GAP_KEYWORDS` → `CareGapAgent`, `HISTORY_KEYWORDS` → `HistoryAgent`
- No keyword match → ALL 3 agents (never empty `agents_used`)
- Hindi detection: `language != "en"` → translate before agent processing

Step 3 (PASS): Run `test_orchestrator.py` → all GREEN

SPRINT 5 BUILD — SUMMARIES + REMINDERS (TDD)

Step 1 (FAIL): Run `tests/functional/test_api_summary.py` → all RED

Step 2 (CODE): Implement `api/routes/summary.py`:

- GET `/api/summary/generate` (NOT POST)
- Returns `StreamingResponse(PDF)` when `total_nodes > 0`
- Returns HTTP 200 + JSON `{"error": "..."} when total_nodes == 0` (NOT HTTP 404 — use 'return dict' not 'raise HTTPException')

Step 3 (PASS): Run test_api_summary.py → GREEN

Step 4 (FAIL): Run tests/functional/test_api_reminders.py → all RED

Step 5 (CODE): Implement api/routes/reminders.py:

- POST /api/reminders/create: reminder_time (str "HH:MM"), days_of_week (list[int] 1-7), RBAC requires "edit"
- GET /api/reminders/list: returns list of reminder dicts
- DELETE /api/reminders/{reminder_id}: soft-delete (is_active=False) returns 404 if id not found

Step 6 (PASS): Run test_api_reminders.py → GREEN

CRITICAL: test_delete_existing_reminder asserts create.status_code == 200

first. If create fails, DELETE test must also fail (no silent skip).

SPRINT 6 BUILD — FRONTEND (TDD with Playwright)

Frontend tech decisions:

- Next.js 14 App Router (not Pages Router) — server components for data fetching
- Tailwind CSS 3.4 — utility-first, no CSS modules
- shadcn/ui — accessible components (Dialog, Table, Badge, Progress)
- React Query (TanStack Query v5) — server state, token refresh handling
- Zustand — client state (user session, notification queue)
- React Hook Form + Zod — form validation with TypeScript inference
- Recharts — medication adherence charts, lab value trends

Frontend directory structure:

```
careorbit-web/src/ |— app/ | |— (auth)/ | |— login/page.tsx | |
|— register/page.tsx | |— (dashboard)/ | |— layout.tsx # Sidebar +
nav with tier badge | |— page.tsx # Health overview (PHIG summary)
```

```

├── ── medications/page.tsx ── ── labs/page.tsx ── ──
reminders/page.tsx ── ── caregivers/page.tsx ── ── chat/page.tsx ──
├── ── settings/ ── ── profile/page.tsx ── ── subscription/page.tsx ──
├── ── api/ ── ── [...proxy]/route.ts # Optional: API proxy for CORS ──
components/ ── ── ui/ # shadcn components ── ── medication-card.tsx ──
├── ── interaction-alert.tsx # Red alert for drug interactions ── ──
confidence-badge.tsx # Color-coded confidence score ── ──
upload-zone.tsx # Drag-and-drop document upload ── ── tier-gate.tsx #
Upgrade prompt for gated features ── ── ad-slot.tsx # Ad container (free
tier only) ── ── lib/ ── ── api.ts # Typed API client + auto token refresh ──
├── ── auth.ts # JWT decode, session management ── ── types.ts #
Generated from OpenAPI spec ── ── tests/ ── ── components/ # Vitest +
RTL component tests ── ── e2e/ # Playwright journey tests

```

Playwright E2E tests to write FIRST:

```
``typescript
```

```
// tests/e2e/ramesh-journey.spec.ts
```

```
import { test, expect } from "@playwright/test";
```

```

test.describe("Ramesh Kumar — Complete Patient Journey", () => {
  test("Step 1: Register as free tier user", async ({ page }) => {
    await page.goto("/register");
    await page.fill('[name="name"]', "Ramesh Kumar");
    await page.fill('[name="email"]', `ramesh.e2e.${Date.now()}@test.com`);
    await page.fill('[name="password"]', "RameshPass123!");
    await page.fill('[name="phone_number"]', "+919876543210");
    await page.click('[type="submit"]');
    await expect(page).toHaveURL("/dashboard");
    // Free tier: ads must be visible
    await expect(page.locator('[data-testid="ad-slot"]')).toBeVisible();
  });
}

```

```

test("Step 2: Upload prescription → see nodes created", async ({ page })
=> {

```

```

// Log in as Ramesh first
await page.goto("/dashboard/medications");
const uploadZone = page.locator('[data-testid="upload-zone"]');
await uploadZone.setInputFiles("tests/fixtures/prescription.jpg");
await expect(page.locator('[data-testid="processing-status"]'))
  .toContainText("success", { timeout: 15000 });
await expect(page.locator('[data-testid="medication-card"]'))
  .toHaveCount(3, { timeout: 5000 });
});

test("Step 3: Interaction alert visible after Ibuprofen upload", async ({ page }) => {
  await page.goto("/dashboard/medications");
  const uploadZone = page.locator('[data-testid="upload-zone"]');
  await
uploadZone.setInputFiles("tests/fixtures/ibuprofen_prescription.jpg");
  // Interaction alert MUST appear — no conditional waiting
  await expect(page.locator('[data-testid="interaction-alert"]'))
    .toBeVisible({ timeout: 15000 });
  await expect(page.locator('[data-testid="interaction-severity"]'))
    .toContainText("HIGH");
});

test("Step 7: Generate health summary PDF", async ({ page }) => {
  const downloadPromise = page.waitForEvent("download");
  await page.click('[data-testid="generate-summary-btn"]');
  const download = await downloadPromise;

  expect(download.suggestedFilename()).toMatch(/CareOrbit_Summary_/);
});
});
```

```

CI/CD PIPELINE — .github/workflows/ci.yml

Generate a GitHub Actions workflow with these EXACT stages in order:

Stage 1 — SAFETY (runs first, blocks all subsequent stages if fails):

- Run tests/false\_positive\_negative/ with -m "critical"
- Run tests/security/test\_cross\_patient\_rbac.py
- Run tests/security/test\_audit\_append\_only.py
- If ANY fail → BLOCK pipeline, alert Slack #clinical-safety channel

Stage 2 — UNIT (runs if Stage 1 passes):

- tests/unit/ with coverage  $\geq 90\%$
- Fail fast: -x flag

Stage 3 — INTEGRATION (runs if Stage 2 passes):

- Spin up Postgres 16 service container with pgcrypto
- Run tests/integration/ with real DB
- Run tests/security/ (refresh token, audit trail, RBAC)
- Coverage gate  $\geq 80\%$  for integration code paths

Stage 4 — FUNCTIONAL (runs if Stage 3 passes):

- Run tests/functional/ with mock Azure services
- Run tests/business\_logic/

Stage 5 — E2E (runs on main branch only):

- Build frontend (next build)
- Start backend + frontend
- Run Playwright tests with --reporter=html
- Upload Playwright report as artifact

Stage 6 — DEPLOY (runs if all stages pass, main branch only):

- Deploy backend to Azure Container Apps
- Deploy frontend to Azure Static Web Apps

- Run smoke tests against production

Generate the complete ci.yml file.

## FINAL DELIVERABLES CHECKLIST

Before declaring implementation complete, verify:

- ☐ pytest runs with zero warnings (not just zero failures)
- ☐ pytest --co (collect-only) shows  $\geq 875$  test items
- ☐ coverage report shows  $\geq 80\%$  overall,  $\geq 90\%$  for security modules
- ☐ No test contains: "if response.status\_code", "if mock.called", "pass" as body
- ☐ All Azure mock paths verified against real SDK import paths
- ☐ Reminder tests: create  $\rightarrow$  hard assert 200/201  $\rightarrow$  delete by real ID
- ☐ Encryption tests: assert call\_count  $\geq 2$  (not if .called)
- ☐ Subscription tests: use standard TestClient (no include\_router hack)
- ☐ Interaction tests: wiring vs algorithm split applied
- ☐ P0 count = 17 (not 14 — v0.3.0 promotions applied)
- ☐ caregiver limit test = @xfail(strict=True) with gate mock, not pass body
- ☐ patient\_confirmed in PHASE1\_SOURCES (not Phase 2)
- ☐ 0.05 floor test ABSENT (was hallucinated, deleted)
- ☐ Playwright E2E: Ramesh journey completes with real interaction alert
- ☐ CI pipeline: Safety stage runs FIRST and blocks on failure
- ☐ README includes: local dev setup, how to run each test stage, Azure resource provisioning guide, environment variables reference